HW1: classifying road obstacles

CMPSC 448

Problem and data

The task is to classify a road obstacle as a pedestrian, a cyclist, or a car using width, height, and speed as features and multiclass logistic regression as the classifier. The inputs are synthetic measurements, not images or collected traffic data. We use PyTorch to drive our experiments.

Each feature is sampled independently within its class from these uniform ranges. Equal-sized intervals have equal probability; a sampled width does not affect height or speed.

Class	Width (m)	Height (m)	Speed (m/s)
Pedestrian	0.3 to 0.9	1.2 to 2.0	0 to 3
Cyclist	0.4 to 1.0	1.0 to 1.9	1 to 10
Car	1.4 to 2.2	1.2 to 2.0	0 to 25

Sampling uses $x = l + u(h - l)$, where $u \sim U(0, 1)$ and l, h are the class-specific bounds. A dictionary groups class names with their ranges; derived PyTorch tensors generate the data in one batch.

Separate, shuffled draws produce 2,400 training examples (800 per class) and 600 test examples (200 per class). Seed 448 fixes data generation and model initialization. Balanced classes give uniform random guessing an expected accuracy of 33.33%.

Each feature is centered and divided by its training standard deviation. The test set uses those same statistics, avoiding test-data leakage.

Model and training

Multinomial logistic regression uses one `nn.Linear(3, 3)` layer: nine weights, three biases, and no hidden layers. For standardized input x,

$$z = Wx + b, \qquad P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{3} e^{z_j}}.$$

`CrossEntropyLoss` minimizes the mean negative log probability of the correct class. It applies log-softmax internally and expects raw scores, so the model does not apply softmax first.

Adam trains for 500 full-batch steps with learning rate 0.05. Each step clears gradients, computes loss, backpropagates, and updates parameters. Prediction selects the largest score; softmax preserves that ordering and is unnecessary for this choice. Test data is used only for evaluation.

Results

The CPU run achieved 93.50% test accuracy: 561 correct predictions out of 600, with final training loss 0.1961.

```
Training examples: 2400 | Test examples: 600
Final training loss: 0.1961
Test accuracy: 93.50% | Chance baseline: 33.33%
Confusion matrix: rows = actual, columns = predicted
              pedestrian     bicycle         car
  pedestrian         191           9           0
     bicycle          30         170           0
         car           0           0         200
```

Figure 1: Browser screenshot of captured script output. Rows are actual classes; columns are predictions.

Pedestrian recall was 95.5% (191/200), cyclist recall was 85.0% (170/200), and car recall was 100% (200/200). All 39 errors confused pedestrians and cyclists: nine pedestrians were labeled cyclists, and 30 cyclists were labeled pedestrians. Their ranges overlap, whereas car widths are separate from both. The perfect car score reflects these invented bounds, not reliable recognition in real traffic.

Challenges and lessons

We standardized the features because speed has a much larger range than width or height. Keeping each class's bounds in one dictionary also made the sampling code easier to follow.

The main lesson was to look beyond overall accuracy. The confusion matrix showed that every mistake involved pedestrians and cyclists, whose ranges overlap. These results describe our synthetic data, not performance on real road measurements.

Run

From the repository root: `uv run cmpsc-448/hw1/main.py`. The script declares Python and PyTorch requirements inline; `uv` installs them in an isolated cache. No GPU or dataset download is required.